

Paradigmas y Lenguajes de Programación 2026

Guía de Estudio — Tema 01

Matías Gel

Ciclo lectivo 2026

Índice de Contenidos

1	Guía de Estudio — Tema 01: Conceptos Introductorios + Intro a TypeScript	2
1.1	¿Cómo usar esta guía?	2
1.2	Objetivos de Aprendizaje	3
1.3	Bloque 1 — ¿Por qué estudiar lenguajes de programación?	3
1.3.1	1.0 ¿Qué es un lenguaje de programación?	3
1.3.2	1.1 ¿Para qué estudiar lenguajes de programación?	4
1.3.3	1.2 El costo de elegir mal un lenguaje	4
1.3.4	1.3 Perspectiva histórica: la línea de tiempo de los lenguajes	4
1.3.5	1.4 Criterios de evaluación de lenguajes — Robert Sebesta	5
1.3.6	1.5 El punto de tensión central: ¿la IA hace obsoletos los lenguajes?	6
1.4	Bloque 2 — Los paradigmas: mapa general	7
1.4.1	2.1 ¿Qué es un paradigma de programación?	7
1.4.2	2.2 La arquitectura de Von Neumann y el paradigma imperativo	7
1.4.3	2.3 El cuello de botella de Von Neumann	8
1.4.4	2.4 La evolución metodológica	8
1.4.5	2.5 Los 4 paradigmas fundamentales	8
1.4.6	2.6 Dominios de aplicación	10
1.5	Bloque 3 — Paradigma imperativo y máquina abstracta	10
1.5.1	3.1 La escalera de abstracciones (Louden & Lambert, Gabbrielli)	10
1.5.2	3.2 La correspondencia Von Neumann → código imperativo	11
1.5.3	3.3 Ejemplo comparativo: el mismo algoritmo en tres niveles	11
1.5.4	3.4 La máquina abstracta — Gabbrielli & Martini	13
1.6	Bloque 4 — Introducción a TypeScript como lenguaje multiparadigma	13
1.6.1	4.1 ¿Por qué TypeScript es el lenguaje del cursado 2026?	13
1.6.2	4.2 TypeScript como ejemplo de máquina intermedia	14
1.6.3	4.3 El mismo problema en tres estilos TypeScript	15
1.6.4	4.4 El sistema de tipos de TypeScript	16
1.6.5	4.5 TypeScript como “acelerador de paradigma”	17
1.7	Bloque 5 — IA Generativa y paradigmas de programación	18
1.7.1	5.1 El cambio de rol del programador	18
1.7.2	5.2 La jerarquía de proficiencia en IA (Schmidt & Runfola, Fig. 12)	18
1.7.3	5.3 La demo en vivo: la IA elige paradigmas	19
1.7.4	5.4 El loop “trust but verify” (Schmidt & Runfola, Fig. 8)	20

1.8	Cierre: Mapa conceptual de la materia	21
1.9	Autoevaluación	21
1.9.1	Q1 — Criterios de Sebesta	21
1.9.2	Q2 — Pipeline de TypeScript	21
1.9.3	Q3 — Paradigma imperativo vs. funcional	22
1.9.4	Q4 — Von Neumann y el paradigma imperativo	22
1.9.5	Q5 — Los 4 paradigmas	22
1.9.6	Q6 — Cuello de botella de Von Neumann	23
1.9.7	Q7 — Trust but verify	23
1.9.8	Q8 — Escalera de abstracciones	24
1.10	Glosario	25
1.11	Referencias bibliográficas	27
1.11.1	Libros de texto principales	27
1.11.2	Artículo de investigación	27

1 Guía de Estudio — Tema 01: Conceptos Introdutorios + Intro a TypeScript

Guía de estudio para el Tema 01, alineada con las filminas y la minuta de clase.

1.1 ¿Cómo usar esta guía?

Esta guía es tu material de referencia para el **Tema 01**. Está organizada siguiendo el mismo orden de la clase, pero con explicaciones más extensas, ejemplos adicionales y actividades de autoevaluación. Podés usarla:

- **Antes de clase:** como lectura preparatoria para saber qué conceptos se van a trabajar.
- **Durante y después de clase:** para profundizar lo visto y aclarar dudas.
- **Para preparar el TP1:** las preguntas del quiz están diseñadas sobre el contenido de esta guía y de la clase.

Materiales fuente complementarios: Durante la preparación de esta guía se utilizaron los siguientes PDFs de referencia bibliográfica que se encuentran en `material/01-conceptos-introdutorios/`: - `01 introduccion.pdf` — Capítulo introductorio (Louden & Lambert) - `012-034.pdf` — Páginas 12–34 del material de Sebesta: criterios de evaluación de lenguajes - `021-044.pdf` — Páginas 21–44: historia y evolución de los LP, paradigmas - `025-184.pdf` — Páginas 25–184: paradigmas, máquinas abstractas, TypeScript

1.2 Objetivos de Aprendizaje

Al finalizar el estudio de este tema, deberías poder:

1. **Justificar** por qué el estudio de paradigmas es relevante en la era de la IA generativa.
 2. **Identificar** los 4 paradigmas fundamentales y sus lenguajes representativos.
 3. **Distinguir** código imperativo de código funcional observando mutación de estado.
 4. **Escribir** una función básica con tipos en TypeScript.
 5. **Aplicar** el loop “trust but verify” para verificar código generado por IA: identificar el paradigma usado, detectar errores semánticos y reformular el prompt con precisión.
-

1.3 Bloque 1 — ¿Por qué estudiar lenguajes de programación?

1.3.1 1.0 ¿Qué es un lenguaje de programación?

Antes de preguntar *por qué* estudiar lenguajes, conviene precisar *qué* son. Tres conceptos se encadenan de lo más simple a lo más específico:

Cómputo > Un proceso que consiste en aplicar una serie de operaciones estructuradas a un conjunto de valores o datos de entrada, con el objetivo de obtener nuevos valores o datos como resultado.

Programa > Una colección definida y ordenada de cálculos diseñada para realizar una tarea específica o resolver un problema concreto. Es la expresión de uno o más cálculos escrita de manera que una máquina pueda ejecutarlos.

Lenguaje de programación > Un conjunto de reglas sintácticas y semánticas usadas para definir programas. Este sistema de notación está compuesto por instrucciones que son comprendidas y ejecutadas por máquinas, usualmente computadoras.

Fuente: Slides cátedra UNTDF 2024 — material/01-conceptos-introductorios/txt/01_introduccion.txt

Aaby (citado en Loudon & Lambert, Cap. 1) agrega una perspectiva más completa:

Una descripción completa de los lenguajes de programación incluye el **modelo computacional**, la **sintaxis** y **semántica** de programas, y las **consideraciones pragmáticas** que dan forma a los lenguajes.

Esta triple dimensión — modelo, sintaxis, semántica — estructura todo el programa de la materia:
- **Modelo computacional** → los paradigmas (Bloques 2–5 y los temas 02–07) - **Sintaxis** → cómo se escriben los programas válidamente (Tema 02) - **Semántica** → qué significan las construcciones y qué efectos producen (Tema 02 en adelante)

1.3.2 1.1 ¿Para qué estudiar lenguajes de programación?

Si ya sabés programar en, digamos, Python o JavaScript, ¿para qué estudiar más lenguajes? La respuesta no es obvia al principio, pero se vuelve clara enseguida: **conocer distintos lenguajes no es saber su sintaxis — es adquirir diferentes modelos mentales para pensar problemas.**

La cantidad de lenguajes existentes es alta (más de 700 con uso documentado, miles de dialectos), y eso no es un accidente ni un caos: cada lenguaje surge para resolver un problema que los anteriores no resolvían bien. Estudiar lenguajes es estudiar *por qué* esas soluciones tomaron la forma que tomaron.

1.3.3 1.2 El costo de elegir mal un lenguaje

La elección del lenguaje de un proyecto tiene consecuencias económicas y técnicas reales:

- Un equipo que elige Node.js para procesamiento numérico intensivo puede terminar reescribiendo todo en Python+NumPy dos años después, con pérdida de código, tiempo y dinero.
- Que pase esto no es incompetencia individual: es falta de conocimiento de paradigmas. Quien eligió Node.js no sabía que el procesamiento vectorial masivo tiene una correspondencia natural con el paradigma funcional y las bibliotecas especializadas del ecosistema científico de Python.

Idea central: La elección del lenguaje determina el modelo mental con el que pensás el problema. No es solo sobre sintaxis — es sobre cómo estructurás la solución.

1.3.4 1.3 Perspectiva histórica: la línea de tiempo de los lenguajes

Los lenguajes no evolucionaron en línea recta. Cada hito respondió a una necesidad concreta:

Año	Lenguaje	¿Qué problema resolvió?
1957	FORTRAN	Reemplazó el ensamblador para cómputo científico — primer lenguaje de alto nivel verdaderamente útil
1960	LISP	Introdujo funciones como datos de primera clase, recursión y evaluación simbólica — base del paradigma funcional
1972	C	Imperativo estructurado con portabilidad sin sacrificar eficiencia — reemplazó el ensamblador en sistemas
1980	Smalltalk	OO puro — Primera implementación pura de orientación a objetos: todo es un objeto, todo es un mensaje

Año	Lenguaje	¿Qué problema resolvió?
1995	Java	OO + máquina virtual → “write once, run anywhere” — desacopló el código del hardware
2008	Python 3	Multiparadigma + ecosistema científico → lenguaje de la IA y la ciencia de datos
2012	TypeScript	Tipos estáticos sobre JavaScript — escalabilidad para proyectos grandes

Patrón: Cada punto en esta línea resuelve un problema del anterior. No es una evolución lineal hacia un lenguaje “perfecto” — es diversificación hacia soluciones especializadas.

Fuente de referencia: [material/01-conceptos-introductorios/021-044.pdf](#) (Louden & Lambert, Cap. 1 — historia y evolución de los LP)

1.3.5 1.4 Criterios de evaluación de lenguajes — Robert Sebesta

Si hay que evaluar un lenguaje, ¿qué se mira? Sebesta sistematizó seis criterios fundamentales:

1.3.5.1 1.4.1 Legibilidad ¿Se puede leer código escrito por otro programador con facilidad?

Un lenguaje legible reduce la carga cognitiva del mantenimiento.

- **Alta legibilidad:** Python — la indentación es parte de la sintaxis, el código se lee casi como inglés.
- **Baja legibilidad:** APL o Perl (con overuse de operadores especiales) — se puede escribir una línea que nadie entiende al día siguiente.

Factores que afectan la legibilidad: - Ortogonalidad (pocos conceptos base usados consistentemente vs. muchas excepciones especiales) - Estructuras de control claras - Tipos de datos expresivos - Buena elección de palabras clave

1.3.5.2 1.4.2 Expresividad ¿El lenguaje facilita expresar lo que querés sin pelear con la sintaxis?

- **Alta expresividad:** SQL para consultas en bases de datos — `SELECT nombre FROM alumnos WHERE nota > 7` es casi lenguaje natural.
- **Baja expresividad:** ensamblador para el mismo tipo de consulta — tenés que manejar punteros, registros y direcciones de memoria.

Hay una tensión frecuente: los lenguajes muy expresivos a veces sacrifican legibilidad al escribir código compacto.

1.3.5.3 1.4.3 Seguridad ¿Las verificaciones del lenguaje (tipado estático, manejo de excepciones, etc.) reducen la posibilidad de errores y vulnerabilidades?

- **Alta seguridad:** TypeScript detecta errores de tipo *antes de ejecutar* — pasar un `string[]` donde se espera `number[]` es un error en tiempo de compilación, no en tiempo de ejecución.
- **Baja seguridad:** JavaScript dinámico — el mismo error silenciosamente retorna NaN en runtime, lo que puede llevar a bugs y comportamientos inseguros.

```
// TypeScript detecta esto ANTES de ejecutar:
function sumaAbs(arr: number[]): number { /* ... */ }
sumaAbs(["hola", "mundo"]);
// Error: Argument of type 'string[]' is not assignable to parameter of type 'number[]'
```

1.3.5.4 1.4.4 Costo Incluye el costo de desarrollo, mantenimiento, entrenamiento del equipo y herramientas. Un lenguaje muy oscuro puede tener bajo costo de desarrollo en el corto plazo, pero alto costo de mantenimiento.

1.3.5.5 1.4.5 Portabilidad ¿El lenguaje funciona en distintas plataformas sin reescribir el código? Java fue diseñado explícitamente para esto: el bytecode corre en cualquier JVM. C también es portátil, pero requiere recompilar por plataforma.

1.3.5.6 1.4.6 Eficiencia Velocidad de ejecución y consumo de memoria. C gana en benchmarks, pero Python es “suficientemente rápido” para la mayoría de aplicaciones y tiene mayor escribibilidad.

La tensión fundamental de Sebesta: No hay lenguaje perfecto. Siempre hay trade-offs entre estos criterios. Python sacrifica eficiencia por legibilidad y escritura; C sacrifica legibilidad por eficiencia. Conocer estos trade-offs es la habilidad central de este tema.

Fuente de referencia: material/01-conceptos-introductorios/012-034.pdf (Sebesta, Cap. 1 §1.3 — criterios de evaluación, pp. 12–34)

1.3.6 1.5 El punto de tensión central: ¿la IA hace obsoletos los lenguajes?

Pregunta provocadora que dejamos planteada al inicio: “¿Importa el lenguaje si la IA puede escribir en cualquiera?”

La respuesta del Bloque 5 lo cierra, pero anticipemos la lógica: precisamente porque la IA puede escribir en cualquier lenguaje, quien la usa necesita saber **verificar** lo que produce. Para verificar código necesitás entender paradigmas. Para decirle a la IA qué querés necesitás el vocabulario correcto. La IA no hace obsoleto el conocimiento — lo transforma en una habilidad de auditoría.

Ejemplo (prompt + regla lógica):

Prompt: "Generá una cláusula de contrato que prohíba compartir datos de clientes fuera del país"
 Regla lógica (Prolog):

```
clausula(Contrato, 'no compartir datos fuera del país') :-
    contiene(Contrato, 'clientes'), contiene(Contrato, 'no compartir').
```

En este esquema, el LLM genera texto y el motor lógico verifica que cumpla la regla; si no, se rechaza y pide reescribir.

1.4 Bloque 2 — Los paradigmas: mapa general

1.4.1 2.1 ¿Qué es un paradigma de programación?

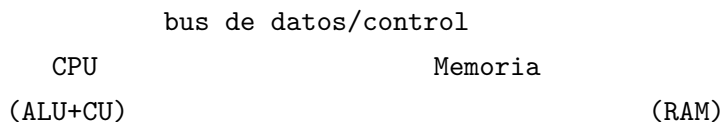
Un paradigma de programación es una **forma de pensar el cómputo**. Es el modelo mental que estructura cómo describís la solución a un problema.

Puntos clave: - **No es solo sintaxis**. Dos lenguajes con sintaxis muy distintas pueden compartir paradigma (Java y C# son ambos OO a pesar de diferencias sintácticas). Un lenguaje puede implementar varios paradigmas (TypeScript puede escribirse de forma imperativa, funcional u OO). - **Es un modelo de cómputo**. El paradigma determina qué constructs usás para expresar la solución: instrucciones secuenciales, funciones matemáticas, relaciones lógicas, objetos que se comunican.

1.4.2 2.2 La arquitectura de Von Neumann y el paradigma imperativo

Los paradigmas no surgieron de la nada — fueron respuestas a restricciones reales de hardware y metodología.

La arquitectura de Von Neumann (1945) describe el modelo de computador que usás hoy:



- La CPU ejecuta instrucciones **una a la vez**, en secuencia.
- Las instrucciones y los datos están en la **misma memoria**.
- La CPU lee una instrucción, la ejecuta, lee la siguiente — ciclo infinito.

El paradigma imperativo es una abstracción directa de este modelo: - Una *variable* mapea directamente a una celda de memoria. - Una *instrucción de asignación* (`x = 5`) es una transferencia CPU → memoria. - Los *saltos de control* (`if`, `while`) corresponden a saltos condicionales del procesador. - El *estado* de un programa = el conjunto de pares (nombre, valor) de todas las

variables en un instante. - Un *cómputo* = una sucesión de estados (cada instrucción modifica el estado).

1.4.3 2.3 El cuello de botella de Von Neumann

Louden & Lambert (Cap. 1) plantean que la ejecución secuencial no es una verdad universal — es una restricción del hardware:

El cuello de botella de Von Neumann es la velocidad del bus entre la CPU y la memoria. Cada instrucción requiere al menos una lectura de memoria. Esto limita el paralelismo natural y el cómputo no determinista.

Esta restricción generó preguntas que dieron origen a otros paradigmas: - ¿Se puede describir cómputo **sin depender de instrucciones secuenciales**? → Sí: el **cálculo lambda** (Church, 1936), base del paradigma funcional. - ¿Se puede describir cómputo como **relaciones y deducciones**? → Sí: la **lógica simbólica** (Robinson, 1965), base del paradigma lógico.

Fuente de referencia: material/01-conceptos-introductorios/01 introduccion.pdf (Louden & Lambert, Cap. 1 — cuello de botella de Von Neumann y surgimiento de los paradigmas)

1.4.4 2.4 La evolución metodológica

Además de las limitaciones de hardware, los paradigmas evolucionaron con las metodologías:

Período	Hito	Consecuencia
Hasta 1970	Programación “artesanal”	Sin metodología sistemática — código espagueti, GOTO ubicuo
Años 70	Análisis y diseño estructurado	Modularización, eliminación del GOTO, funciones y procedimientos
Años 80	Abstracción de datos (Simula, Ada)	El dato y sus operaciones juntos — precursor del OO
Años 80	Programación funcional madura (ML, Haskell)	Sin estado mutable, basado en lambda calculus
Años 90	OO mainstream (Java)	Objetos como unidad fundamental del programa
2000s+	Multiparadigma (Python, TypeScript, Scala)	Sin compromiso obligatorio con un solo modelo

1.4.5 2.5 Los 4 paradigmas fundamentales

Paradigma	Base formal	Unidad de abstracción	Modelo de estado	Lenguajes representativos
Imperativo	Arquitectura Von Neumann	Instrucción	Mutable (explícito)	C, Go, Pascal, FORTRAN
Orientado a Objetos	Imperativo + encapsulamiento	Objeto / mensaje	Mutable (encapsulado)	Java, C#, Dart
Declarativo (Funcional)	Cálculo lambda (Church, 1936)	Función matemática	Inmutable (sin efectos laterales)	Haskell, Clojure, LISP/Scheme, F#
Declarativo (Lógico)	Lógica simbólica (Robinson, 1965)	Relación / hecho / consulta	Sin concepto de estado	Prolog

1.4.5.1 Notas sobre cada paradigma: Imperativo: El más cercano al hardware. Un programa es una secuencia de instrucciones que modifican el estado. El estado es el conjunto de variables y sus valores en un momento dado.

Orientado a Objetos: Es una extensión del imperativo, no una ruptura. Agrega encapsulamiento (el objeto oculta su estado interno) y paso de mensajes (los objetos se comunican llamando a sus métodos). El estado sigue siendo mutable, pero está encapsulado dentro de los objetos.

Funcional (declarativo): Se basa en el cálculo lambda de Alonzo Church (1936) — antes de que existieran las computadoras. Un programa describe *qué* transformar (funciones matemáticas puras), no *cómo* ejecutar los pasos; no hay variables mutables ni efectos laterales. El mismo input siempre produce el mismo output.

Lógico (declarativo): Un programa declara hechos y reglas. La ejecución es una búsqueda de pruebas. En lugar de decirle al computador cómo resolver algo, se expresa qué condiciones deben cumplirse y el motor de inferencia encuentra las soluciones.

Multiparadigma: TypeScript, Python, Scala, F# no se comprometen con un solo modelo. El mismo archivo puede tener código imperativo, funcional y OO. La flexibilidad es poderosa, pero también implica más responsabilidad de estilo en equipos.

Fuente de referencia: material/01-conceptos-introductorios/021-044.pdf y material/01-conceptos-introductorios/045-048.pdf (Louden & Lambert + Gabbrielli & Martini — paradigmas fundamentales)

1.4.6 2.6 Dominios de aplicación

El paradigma correcto depende del dominio del problema:

Paradigma	Dominio de aplicación ideal	Lenguajes típicos
Imperativo	Sistemas operativos, drivers, embebidos, rendimiento crítico	C, Go, Rust
OO	Aplicaciones empresariales, GUIs, modelado de dominio complejo	Java, C#, Kotlin
Funcional	Finanzas de alta confiabilidad, compiladores, transformaciones de datos, IA	Haskell, Clojure, Scala
Lógico	IA simbólica, sistemas expertos, NLP, verificación formal	Prolog, Mercury
Multiparadigma	Web full-stack, APIs, scripts, ciencia de datos	TypeScript, Python

1.5 Bloque 3 — Paradigma imperativo y máquina abstracta

Antes de continuar: Si saltaste las secciones 2.2 y 2.3 (Von Neumann y el cuello de botella), volvé a leerlas antes de seguir con esta sección. Sin ese marco, los ejemplos de código de abajo pierden su sentido pedagógico — no se trata de aprender C, sino de ver cómo el hardware de los años 40 determina el paradigma imperativo que usamos hoy.

1.5.1 3.1 La escalera de abstracciones (Louden & Lambert, Gabbrielli)

Todo lenguaje ocupa un nivel en una jerarquía de abstracción:

Frameworks (React, Django, Rails)	← Nivel 3: abstracción de plataforma
↑	
Java, Python, TypeScript	← Nivel 2: GC, tipos, ecosistema
↑	
C, Pascal, Go	← Nivel 1: estructura, tipos básicos, funciones
↑	
LC-3, ensamblador x86	← Nivel 0: registros, saltos, direcciones de memoria
↑	
Lenguaje de máquina (bits)	← Nivel -1: binario puro

Al **subir** se gana: - Legibilidad (código más expresivo, más cercano al problema) - Escribibilidad (menos código para expresar lo mismo) - Portabilidad (menos dependencia del hardware)

Al **bajar** se gana: - Control (acceso directo al hardware) - Eficiencia (sin overhead del intérprete/compilador/GC)

La pregunta que guía esta materia: “¿Qué abstraje? ¿Qué perdí?”

Cuando escribís `arr.map(x => Math.abs(x))` en TypeScript, abstraíste el loop, el índice, la variable acumuladora, los saltos condicionales. Lo que “perdiste” es visibilidad del estado intermedio. Puede importar o no, según el contexto.

Fuente de referencia: material/01-conceptos-introdutorios/01 introduccion.pdf (Louden & Lambert, Cap. 1 — Fig. 1.4 y 1.5, escalera de abstracciones)

1.5.2 3.2 La correspondencia Von Neumann → código imperativo

Esta tabla ilustra la correspondencia directa entre hardware y lenguaje imperativo:

Componente de Von Neumann	Concepto en lenguaje imperativo
Celda de memoria con dirección	Variable con nombre
Transferencia de valor CPU → memoria	Instrucción de asignación (<code>x = 5</code>)
Transferencia de valor memoria → CPU	Lectura de variable (<code>y = x + 1</code>)
Salto condicional del procesador	<code>if (...) { ... } else { ... }</code>
Salto incondicional	<code>goto</code> , <code>break</code> , <code>continue</code>
Contador de programa (PC)	Flujo de control explícito
Conjunto de valores de celdas en un instante	Estado del programa
Sucesión de instrucciones ejecutadas	Cómputo imperativo

Consecuencia importante: El paradigma imperativo no es una convención arbitraria de programadores. Es una **abstracción directa del hardware de los años 40**. Todo programa imperativo que escribís es, en el fondo, un conjunto de instrucciones de transferencia de datos entre CPU y memoria.

1.5.3 3.3 Ejemplo comparativo: el mismo algoritmo en tres niveles

El objetivo: calcular la suma de los valores absolutos de un array de 10 enteros.

1.5.3.1 Nivel 0 — Ensamblador LC-3 LC-3 es el ensamblador usado en los cursos introductorios de arquitectura de computadoras. Este algoritmo requiere 13 instrucciones:

```
; Suma valores absolutos - LC-3
; R0 = acumulador, R1 = puntero al array, R2 = contador
    AND R0, R0, #0      ; acc = 0 (limpiar R0)
```

```

        LEA R1, DATA      ; R1 = dirección de inicio del array
        AND R2, R2, #0
        ADD R2, R2, #10    ; R2 = 10 (contador de iteraciones)
LOOP    LDR R3, R1, #0     ; R3 = memoria[R1] (cargar elemento actual)
        BRzp POS          ; si R3 >= 0, saltar a POS (no negar)
        NOT R3, R3         ; negar: complemento a 2 paso 1
        ADD R3, R3, #1     ; negar: complemento a 2 paso 2
POS     ADD R0, R0, R3     ; acc += abs(elemento)
        ADD R1, R1, #1     ; avanzar puntero al siguiente elemento
        ADD R2, R2, #-1    ; decrementar contador
        BRp LOOP          ; si R2 > 0, repetir el loop
DATA    .FILL 3           ; array: 3, -1, 4, -1, 5, ...

```

Observaciones: - El acumulador es el **registro R0** — una celda física de la CPU. - El puntero R1 se incrementa *manualmente* una instrucción a la vez. - El valor absoluto se calcula con complemento a 2 (dos instrucciones). - Sin comentarios, este código es ilegible para quien no lo escribió.

```

int suma_abs(int arr[], int n) {
    int acc = 0;
    for (int i = 0; i < n; i++)
        acc += (arr[i] < 0) ? -arr[i] : arr[i];
    return acc;
}

```

1.5.3.2 Nivel 1 — C (imperativo de alto nivel) ¿Qué ganamos? - `int acc` reemplaza el registro R0 — legible, con nombre descriptivo. - El `for` reemplaza el contador, el decremento y el salto — una línea vs. cinco. - El operador ternario `? :` reemplaza el `BRzp` y el `NOT/ADD` — una expresión vs. cuatro instrucciones. - El código tiene 5 líneas vs. 13 instrucciones de ensamblador.

¿Qué sigue implícito (lo que “perdimos” al subir)? - La gestión de punteros (`arr[]` es un puntero a memoria bajo el capó). - La ubicación en memoria de `acc` e `i`. - Los registros de la CPU que el compilador decide usar. - Los efectos de caché (cuándo y cómo la CPU lee de memoria).

Sigue siendo imperativo: `acc` es mutable, el loop modifica el estado paso a paso.

```

const sumaAbs = (arr: number[]): number =>
    arr.map(x => Math.abs(x))
        .reduce((acc, x) => acc + x, 0);

```

```
console.log(sumaAbs([3, -1, 4, -1, 5])); // → 14
```

1.5.3.3 Nivel 2 — TypeScript (funcional) ¿Qué cambió? - Sin `let acc = 0` — no hay variable mutable acumuladora visible. - Sin loop explícito — el recorrido está abstraído dentro de `map` y `reduce`. - `map(x => Math.abs(x))` transforma cada elemento en su valor absoluto → `[3, 1, 4, 1, 5]`. - `reduce((acc, x) => acc + x, 0)` suma todos los elementos. - Estilo funcional: composición de funciones, sin mutación de estado observable.

1.5.4 3.4 La máquina abstracta — Gabbrielli & Martini

Gabbrielli & Martini (Cap. 1) formalizan el concepto central: **todo lenguaje define una máquina abstracta**.

Definición: Una máquina abstracta M_L es un ejecutor de algoritmos definido por el lenguaje L . Incluye un intérprete que ejecuta los programas del lenguaje y las estructuras de datos necesarias para la ejecución.

No hay lenguaje sin máquina abstracta — escribir en un lenguaje es programar esa máquina.

Dos formas de implementar M_L sobre una máquina física:

Interpretación pura:

```
programa L → [intérprete en runtime] → resultado
Flexible (el intérprete puede cambiar en runtime)
Más lento (overhead de interpretación en cada instrucción)
```

Compilación pura:

```
programa L → [compilador] → programa objeto (lenguaje máquina)
Rápido (ya está traducido al lenguaje nativo)
Menos flexible, dependiente de la arquitectura target
```

En la práctica: siempre existe un **lenguaje intermedio** entre el lenguaje de alto nivel y la máquina física. La realidad de la implementación es una combinación de los dos modelos puros.

Fuente de referencia: [material/01-conceptos-introductorios/025-184.pdf](#) (Gabbrielli & Martini, Cap. 1 — máquinas abstractas, pp. 25–44)

1.6 Bloque 4 — Introducción a TypeScript como lenguaje multiparadigma

1.6.1 4.1 ¿Por qué TypeScript es el lenguaje del cursado 2026?

Razón	Detalle
Multiparadigma	Podés escribir el mismo problema en estilo imperativo, funcional u OO en el mismo archivo — ideal para comparar paradigmas
Tipos estáticos	El compilador detecta errores antes de ejecutar → alta confiabilidad (criterio Sebesta)
Ecosistema	npm tiene más de 2.3 millones de paquetes; corre en frontend, backend (Node.js, Deno, Bun) y IA tooling
Adopción real	Lenguaje #1 o #2 en encuestas de desarrolladores 2023–2025 (Stack Overflow)
Relevancia con IA	Los modelos de lenguaje generan TypeScript con alta fidelidad — fácil auditar con conocimiento del paradigma

TypeScript reemplaza a Kotlin en el cursado 2026. Cumple los mismos objetivos conceptuales (multiparadigma, tipado estático, moderno) con mayor adopción en el mercado actual.

1.6.2 4.2 TypeScript como ejemplo de máquina intermedia

TypeScript es la demostración viva de la teoría de Gabbrielli. El pipeline de ejecución es:

archivo.ts

```
↓ tsc (TypeScript Compiler)
    verifica tipos en tiempo de compilación
    transpila a JavaScript puro
```

archivo.js ← LENGUAJE INTERMEDIO

```
↓ V8 / Node.js / Deno (JIT compiler)
    compila JS a código máquina nativo en runtime
```

Ejecución en CPU

Comparado con otros lenguajes:

Lenguaje fuente	Compilador	Intermediario	Runtime
TypeScript	tsc	.js (JavaScript)	V8 / Node.js / Deno
Java	javac	.class (bytecode JVM)	JVM (HotSpot)
Python	CPython (implícito)	bytecode .pyc	CPython VM
C	gcc / clang	código objeto .o	CPU (nativo)
Haskell	GHC	Core + LLVM IR	LLVM / nativo

Ninguno de estos lenguajes “se interpreta” ni “se compila” en sentido puro.

Todos usan máquina intermedia — exactamente como predice Gabbrielli & Martini Cap.

1.

¿Por qué importa el pipeline para TypeScript? - `tsc` es el verificador de tipos → la confiabilidad ocurre aquí. - V8 es el motor que ejecuta el código → las optimizaciones de rendimiento ocurren aquí. - Son dos preocupaciones separadas, en dos etapas separadas.

1.6.3 4.3 El mismo problema en tres estilos TypeScript

Los tres ejemplos resuelven el mismo problema: sumar los valores absolutos de `[3, -1, 4, -1, 5]`. Todos producen 14.

```
function sumaAbs(arr: number[]): number {
  let acc = 0;
  for (let i = 0; i < arr.length; i++) {
    acc += arr[i] < 0 ? -arr[i] : arr[i];
  }
  return acc;
}

console.log(sumaAbs([3, -1, 4, -1, 5])); // → 14
```

1.6.3.1 Estilo imperativo Marcadores de paradigma imperativo: - `let acc = 0` — variable mutable - El loop `for` modifica `acc` y `i` en cada iteración — estado que cambia - El cómputo *es* la sucesión de estados de `acc`: `0 → 3 → 4 → 8 → 9 → 14`

```
const sumaAbs = (arr: number[]): number =>
  arr.map(x => Math.abs(x))
    .reduce((acc, x) => acc + x, 0);

console.log(sumaAbs([3, -1, 4, -1, 5])); // → 14
```

1.6.3.2 Estilo funcional Marcadores de paradigma funcional: - Sin `let` — sin variables mutables - `map` es una **función de orden superior** (recibe una función como argumento) - `reduce` también es función de orden superior - El mismo patrón existe en Scheme (LISP, 1960): `(reduce + (map abs lista))` — tiene 60 años - No hay loop explícito, no hay índice, no hay estado visible

```
class Calculadora {
  sumaAbs(arr: number[]): number {
    return arr.reduce((a, x) => a + Math.abs(x), 0);
  }
}

const calc = new Calculadora();
console.log(calc.sumaAbs([3, -1, 4, -1, 5])); // → 14
```

1.6.3.3 Estilo OO Marcadores de paradigma OO: - Los datos y el comportamiento están encapsulados en la clase `Calculadora` - El objeto `calc` recibe un mensaje (`sumaAbs`) con argumentos - La implementación interna usa `reduce` (funcional) — esto es multiparadigma

Los tres producen 14. La elección es del programador. La elección afecta la legibilidad, testabilidad y consistencia del equipo.

1.6.4 4.4 El sistema de tipos de TypeScript

TypeScript agrega a JavaScript una capa de tipos estáticos. Es una de sus ventajas principales.

```
// Anotaciones explícitas
let nombre: string = "Paradigmas";
let año: number = 2026;
let activo: boolean = true;

// Inferencia - TypeScript deduce el tipo a partir del valor
let materia = "Paradigmas"; // tipo inferido: string
let version = 5;             // tipo inferido: number
```

1.6.4.1 Anotaciones explícitas e inferencia

```
function saludar(nombre: string): string {
  return `Hola, ${nombre}!`;
}

// Arrow function (lambda) tipada
const cuadrado = (x: number): number => x * x;
```


1.6.4.2 Funciones tipadas

```
const notas: number[] = [8, 9, 7, 10];

interface Alumno {
  nombre: string;
  legajo: number;
  notas: number[];
  promedio?: number; // el ? indica campo opcional
}

const alumno: Alumno = {
  nombre: "Ana García",
  legajo: 12345,
  notas: [8, 9, 10]
};
```

1.6.4.3 Arrays e interfaces

```
// El compilador detecta esto ANTES de ejecutar:
function sumaAbs(arr: number[]): number {
  return arr.reduce((a, x) => a + Math.abs(x), 0);
}

sumaAbs(["hola", "mundo"]);
// Error TS2345: Argument of type 'string[]' is not
// assignable to parameter of type 'number[]'.
```

1.6.4.4 Detección de errores en tiempo de compilación

Este es el criterio de **confiabilidad** (Sebesta §1.3) en acción. El compilador es el primer revisor de tu código — detecta una clase completa de bugs antes de que el programa llegue a ejecutarse.

1.6.5 4.5 TypeScript como “acelerador de paradigma”

TypeScript no obliga a ningún paradigma — puede usar los tres. Esta flexibilidad tiene consecuencias:

- **Ventaja:** cada proyecto puede elegir el estilo más apropiado para cada problema.
- **Desventaja:** sin disciplina de equipo o linting, el mismo codebase puede mezclar estilos de forma inconsistente.

A lo largo de la materia vamos a escribir el mismo tipo de problema en diferentes estilos para comparar las consecuencias prácticas de cada elección.

1.7 Bloque 5 — IA Generativa y paradigmas de programación

1.7.1 5.1 El cambio de rol del programador

Schmidt & Runfola (2025, arXiv:2511.17696) documentaron un cambio fundamental en la distribución del tiempo de trabajo de los programadores:

Actividad	Antes (pre-IA)	Hoy (con IA generativa)
Codificación manual + depuración	70%	20%
Prompting, supervisión y orquestación de IA	0%	50%
Comprensión y formulación del problema	30%	30%

“Natural language has become the new compiler, and developer’s focus is migrating from syntax and semantics to strategy.” — Schmidt & Runfola (2025)

La IA no elimina la necesidad de saber programar — la transforma. El trabajo intelectual ahora es:
 - Formulación precisa del problema - Diseño del prompt correcto - Verificación semántica del output (¿hizo lo que pedí? ¿usó el paradigma que quería?) - Detección de errores sutiles que la IA no señala

1.7.2 5.2 La jerarquía de proficiencia en IA (Schmidt & Runfola, Fig. 12)

AI MASTERY

Construir, optimizar y auditar sistemas de IA

Requiere: matemática, ML, sistemas distribuidos

→ Investigadores e ingenieros de IA

AI FLUENCY

← USTEDES

Diseñar prompts, criticar modelos, construir soluciones

Requiere: pensamiento computacional, paradigmas, semántica

→ Desarrolladores competentes en la era de la IA

AI LITERACY

Leer e interpretar lo que genera la IA
→ Cualquier usuario con educación básica

El argumento central: AI Fluency sin conocimiento de paradigmas es imposible. Si la IA genera código funcional donde vos querías imperativo, y no sabés distinguir uno del otro, no podés detectarlo. Si le pedís a la IA que explique qué máquina ejecuta tu código y ella comete un error, no podés identificar el error si no conocés la respuesta correcta.

1.7.3 5.3 La demo en vivo: la IA elige paradigmas

En clase se realizó una demo con tres prompts a un modelo de IA (Copilot o Claude):

1.7.3.1 Prompt 1 — Sin restricción de paradigma

"Escribí en TypeScript una función que calcule los primeros N números de la serie de Fibonacci."

Output típico de la IA:

```
function fibonacci(n: number): number[] {  
    let result: number[] = [];  
    let a = 0, b = 1;  
    for (let i = 0; i < n; i++) {  
        result.push(a);  
        [a, b] = [b, a + b];  
    }  
    return result;  
}
```

Análisis: let result, let a, let b mutables + loop for = **paradigma imperativo**. La IA tiende al paradigma más difundido entre sus datos de entrenamiento.

1.7.3.2 Prompt 2 — Restricción funcional explícita

"Implementá lo mismo (serie de Fibonacci) en estilo funcional puro, sin mutación de estado, sin variables intermedias."

Output esperado:

```
const fib = (n: number): number =>  
    n <= 1 ? n : fib(n - 1) + fib(n - 2);  
const fibonacci = (n: number): number[] =>  
    Array.from({ length: n }, (_, i) => fib(i));
```

Análisis: Sin `let`, sin `loop`, recursión pura. Si la IA usa `let` igual después de esta instrucción, es evidencia de que no interpretó correctamente la restricción del prompt — y solo lo detectás si sabés qué es el paradigma funcional.

1.7.3.3 Prompt 3 — Verificación de comprensión de máquinas abstractas

"Escribí en TypeScript una función que calcule los primeros N números de la serie de Fibonacci usando ``let`` y un bucle.
Luego explicá qué máquina abstracta ejecuta ese código."

Respuesta correcta esperada (verificar estos puntos): - `tsc` compila `.ts` → `.js` (lenguaje intermedio)
- V8 / Node.js / Deno ejecuta el `.js` resultante vía JIT - TypeScript no se ejecuta directamente — siempre hay máquina intermedia - Comparación con JVM (Java) o CPython (Python)

Si la IA omite alguno de estos puntos, tenés un gap concreto del “trust but verify” en acción.

1.7.4 5.4 El loop “trust but verify” (Schmidt & Runfola, Fig. 8)

El flujo de trabajo correcto con IA generativa para programación:

1. Formular el problema con precisión
↓
 2. Hacer el prompt a la IA con restricciones explícitas
↓
 3. Revisar el output con conocimiento de dominio
 - ¿Qué paradigma usó?
 - ¿La semántica es correcta?
 - ¿Hay mutación de estado no deseada?
 ↓
 4. Testear con casos borde (arrays vacíos, negativos, grandes)
↓
 5. Refinar el prompt o escribir manualmente si falla
↑
- (repetir hasta obtener el resultado correcto)

El “sweet spot”: Schmidt & Runfola (Fig. 14) muestran que la dependencia excesiva en IA atrofia habilidades cognitivas. Los fundamentos — paradigmas, semántica, máquinas abstractas — son el antídoto. Cuanto más sólido sea tu modelo mental, más eficiente y seguro es tu uso de la IA.

1.8 Cierre: Mapa conceptual de la materia

El Tema 1 establece el mapa de toda la cursada:

TEMA 1 - Fundamentos: paradigmas, lenguajes, IA, TypeScript intro

TEMA 2 - Sintaxis y semántica (Louden & Lambert, Cap. 1 §1.4)

↓

TEMAS 3-6 - Paradigma funcional (TypeScript + Python)

TEMA 7 - Paradigma lógico (Prolog)

TEMA 8 - Paradigma OO (TypeScript)

↓

TEMAS 9-14 - Conceptos transversales:

variables · tipos · control · módulos · polimorfismo

↓

TEMA 15 - Concurrencia y paralelismo

1.9 Autoevaluación

Estas preguntas te permiten verificar si comprendiste los conceptos clave del tema. Respondé sin mirar las respuestas, luego verificá.

1.9.1 Q1 — Criterios de Sebesta

¿Cuál de los 6 criterios de Sebesta describe mejor la capacidad de TypeScript de detectar `Argument of type 'string[]' is not assignable to parameter of type 'number[]'` antes de ejecutar?

Ver respuesta

Confiabilidad. El sistema de tipos estáticos reduce la posibilidad de bugs verificando los tipos en tiempo de compilación, antes de que el programa llegue a ejecutarse. La legibilidad se refiere a la lectura del código, la escribibilidad a la facilidad de expresar intenciones — ninguna de esas es la capacidad de detectar errores antes de runtime.

1.9.2 Q2 — Pipeline de TypeScript

Completá el pipeline: `archivo.ts` → `[_]` → `archivo.js` → `[_]` → ejecución en CPU

Ver respuesta

archivo.ts → [tsc (TypeScript Compiler)] → archivo.js → [V8 / Node.js / Deno (JIT)] → ejecución en CPU

- **tsc** es el compilador que verifica tipos y genera JavaScript.
 - V8 / Node.js / Deno compilan JS a código máquina nativo en runtime (JIT = Just-In-Time compilation).
-

1.9.3 Q3 — Paradigma imperativo vs. funcional

Identificá el paradigma de cada fragmento y justificá la respuesta en una oración:

```
// Fragmento A
let total = 0;
for (const x of datos) total += Math.abs(x);

// Fragmento B
const total = datos.map(Math.abs).reduce((a, b) => a + b, 0);
```

Ver respuesta

- **Fragmento A: imperativo.** La variable `total` es mutable y se modifica instrucción a instrucción — el cómputo es una sucesión de estados.
 - **Fragmento B: funcional.** No hay variables mutables ni loop explícito; el resultado se obtiene por composición de funciones de orden superior (`map` y `reduce`).
-

1.9.4 Q4 — Von Neumann y el paradigma imperativo

¿A qué concepto de Von Neumann corresponde directamente una “variable” en un lenguaje imperativo?

Ver respuesta

A una **celda de memoria con nombre** (en Von Neumann, direccionada numéricamente). La instrucción de asignación `x = 5` es una transferencia de valor CPU → memoria. El “estado” del programa es el conjunto de todos esos pares (nombre, valor) en un instante dado.

1.9.5 Q5 — Los 4 paradigmas

Para cada enunciado, indicá a qué paradigma corresponde:

Enunciado	Paradigma
“Un programa es un conjunto de hechos y reglas; la ejecución es una búsqueda de pruebas”	?
“Todo es un objeto; los objetos se comunican mediante mensajes”	?
“Un programa es una composición de funciones puras; sin estado mutable”	?
“Un programa es una secuencia de instrucciones que modifican el estado”	?

Ver respuesta

Paradigma

Lógico (Prolog)

Orientado a Objetos (Java)

Funcional (Haskell, LISP)

Imperativo (C, Go, Pascal)

1.9.6 Q6 — Cuello de botella de Von Neumann

Según Louden & Lambert (Cap. 1), ¿entre qué dos componentes de hardware se produce el cuello de botella de Von Neumann? ¿Por qué esta limitación motivó el surgimiento del paradigma funcional?

Ver respuesta

El cuello de botella se produce en el **bus entre la CPU y la memoria (RAM)**. Cada instrucción del modelo secuencial requiere al menos un acceso a memoria, lo que limita la velocidad de cómputo y dificulta el paralelismo.

Esto motivó la pregunta: ¿se puede describir cómputo sin depender de instrucciones secuenciales? La respuesta fue sí: el **cálculo lambda** de Church (1936) modela el cómputo como transformación de funciones, sin referenciar celdas de memoria ni ejecución secuencial.

1.9.7 Q7 — Trust but verify

Nota: Esta pregunta ejercita el mismo razonamiento que **P05 del TP1**. Hay preguntas del quiz que requieren recordar lo que ocurrió en la demo en vivo de clase — no alcanzan

la guía sola. Si no estuviste en clase, asegurate de leer bien la sección 5.3 de esta guía.

Un compañero usa este prompt y obtiene el resultado de abajo. ¿Es correcto el resultado? Si hay un problema, ¿cuál es?

Prompt: “Escribí una función TypeScript funcional pura que calcule el cuadrado de cada número en un array”

Resultado de la IA:

```
function cuadrados(arr: number[]): number[] {  
  let result: number[] = [];  
  for (const x of arr) {  
    result.push(x * x);  
  }  
  return result;  
}
```

Ver respuesta

No es correcto según el prompt. El prompt pidió estilo **funcional puro**, pero la IA generó código **imperativo**:

- `let result = []` — variable mutable
- `result.push(...)` — mutación del array dentro del loop
- Loop `for...of` explícito

La versión funcional correcta sería:

```
const cuadrados = (arr: number[]): number[] =>  
  arr.map(x => x * x);
```

Sin `let`, sin mutación, sin loop explícito. Esta es exactamente la situación del “trust but verify”: sin conocer la diferencia entre imperativo y funcional, no podés detectar que la IA no respetó la restricción del prompt.

1.9.8 Q8 — Escalera de abstracciones

¿Qué se gana y qué se pierde al subir de C a TypeScript en la escalera de abstracciones?

Ver respuesta

Se gana: - Mayor legibilidad (funciones de orden superior, tipos expresivos) - Mayor escribibilidad (menos código para el mismo resultado) - Mayor seguridad de tipos (TypeScript detecta más errores)

en compilación que C) - Mayor portabilidad (el mismo TypeScript corre en Node, Deno, browser)

Se pierde: - Control directo sobre la memoria (no podés gestionar punteros) - Eficiencia (el overhead del compilador tsc + V8/JIT es mayor que C $\rightarrow$ CPU) - Visibilidad del estado intermedio (los efectos de **reduce** no son explícitos)

1.10 Glosario

Término	Definición
Paradigma de programación	Modelo mental que estructura cómo se describe la solución a un problema. Define las abstracciones disponibles y la forma de componer el programa.
Paradigma imperativo	Modelo en que un programa es una secuencia de instrucciones que modifican el estado del sistema. Cada instrucción es un cambio de estado.
Paradigma funcional	Modelo basado en el cálculo lambda. Un programa es una composición de funciones matemáticas puras. Sin estado mutable ni efectos laterales.
Paradigma orientado a objetos	Extensión del imperativo donde la unidad fundamental es el objeto — encapsula estado y comportamiento. Los objetos se comunican mediante mensajes.
Paradigma lógico	Un programa es un conjunto de hechos y reglas. La ejecución es una búsqueda de pruebas (deducción). No hay concepto de estado.
Multiparadigma	Un lenguaje que soporta varios paradigmas simultáneamente y no obliga a elegir uno. Ej: TypeScript, Python, Scala.
Legibilidad	Criterio Sebesta: facilidad con que un programador puede leer y comprender código escrito por otros.
Escribibilidad	Criterio Sebesta: facilidad para expresar la solución a un problema con el lenguaje.
Confiabilidad	Criterio Sebesta: capacidad del lenguaje de detectar y prevenir errores (tipos estáticos, verificación formal).
Máquina abstracta	Un ejecutor de algoritmos definido por un lenguaje L. Todo lenguaje define una máquina abstracta implícita. (Gabbrielli & Martini)

Término	Definición
Lenguaje intermedio	Representación intermedia generada durante la traducción de un lenguaje de alto nivel al lenguaje de máquina. Ej: JavaScript (para TypeScript), bytecode JVM (para Java).
Compilación	Traducción del programa fuente completo a lenguaje objeto antes de la ejecución.
Interpretación	Ejecución del programa fuente instrucción a instrucción por un intérprete en runtime.
JIT (Just-In-Time)	Compilación dinámica durante la ejecución — el intérprete compila fragmentos de código a código nativo la primera vez que los ejecuta. Usado por V8 (Node.js/Deno).
Cuello de botella de Von Neumann	La limitación de velocidad del bus CPU-memoria en la arquitectura de Von Neumann. Impide el paralelismo de instrucciones a nivel arquitectural.
Estado	Conjunto de pares (nombre, valor) de todas las variables de un programa en un instante dado. El paradigma imperativo define el cómputo como sucesión de estados.
Función de orden superior	Función que recibe funciones como argumento y/o retorna funciones. Ej: <code>map</code> , <code>reduce</code> , <code>filter</code> . Base del paradigma funcional.
Cálculo lambda	Sistema formal creado por Alonzo Church (1936) para describir la computación mediante funciones. Base matemática del paradigma funcional.
Inferencia de tipos	Capacidad del compilador de deducir el tipo de una expresión sin que el programador lo declare explícitamente.
AI Fluency	Nivel de competencia con IA que permite diseñar prompts, criticar el comportamiento del modelo y construir soluciones completas. Requiere pensamiento computacional y conocimiento de paradigmas. (Schmidt & Runfola)
Trust but verify	Metodología de trabajo con IA: generar código con el modelo, luego verificar manualmente la corrección semántica y el paradigma usado antes de confiar en el resultado.

Término	Definición
tsc	TypeScript Compiler — herramienta oficial que convierte código TypeScript a JavaScript. Realiza la verificación de tipos en esta etapa.
V8	Motor JavaScript de Google (usado en Chrome, Node.js, Deno). Compila JavaScript a código máquina nativo mediante JIT.

1.11 Referencias bibliográficas

Los contenidos de este tema están basados en los siguientes materiales, disponibles como PDF en [material/01-conceptos-introdutorios/](#):

1.11.1 Libros de texto principales

Sebesta, Robert W. *Concepts of Programming Languages*, 12th edition. Pearson, 2018. — Sección cubierta: Capítulo 1 (criterios de evaluación de lenguajes, historia de los LP). — Archivo: [material/01-conceptos-introdutorios/012-034.pdf](#) (pp. 12–34)

Louden, Kenneth C. y Lambert, Kenneth A. *Programming Languages: Principles and Practice*, 3rd edition. Course Technology, 2012. — Secciones cubiertas: Capítulo 1 (paradigmas, cuello de botella Von Neumann, escalera de abstracciones, funciones de orden superior). — Archivos: [material/01-conceptos-introdutorios/01_introduccion.pdf](#) (Cap. 1 completo) · [material/01-conceptos-introdutorios/021-044.pdf](#) (pp. 21–44)

Gabbrielli, Maurizio y Martini, Simone. *Programming Languages: Principles and Paradigms*, 2nd edition. Springer, 2010. — Secciones cubiertas: Capítulo 1 (máquinas abstractas, interpretación pura, compilación pura, lenguajes intermedios). — Archivo: [material/01-conceptos-introdutorios/025-184.pdf](#) (pp. 25–44 del Cap. 1)

1.11.2 Artículo de investigación

Schmidt, Douglas C. y Runfola, Dan. *Liberating Logic in the Age of AI: Going Beyond Programming with Computational Thinking*. arXiv:2511.17696, 2025. — Usado en: Bloque 5 (cambio de rol del programador, jerarquía de proficiencia AI Literacy/Fluency/Mastery, loop trust-but-verify, Fig. 8, 12 y 14).

Guía generada por Dra. Sofía (study-guide-writer) · Paradigmas y Lenguajes de Programación 2026
· *UNTDF / IDEI*